

Real-world (rebuild Project 2 as a graph) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangGraph, plus proving equivalence with your original loop, not just "it runs"). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Doc07's loop was: ask the model what to do (think), run the tool it picked (act), feed the result back in (observe), repeat. That's a `while` loop with two real steps inside it. As a graph, those two steps become two nodes — and the "repeat" part becomes an edge that goes back to the first node instead of forward to a new one.

For this exercise: a "think" node (calls the model, same as your Doc07 loop), an "act" node (runs whichever tool the model picked), a conditional edge after "think" that either goes to "act" or finishes, and a plain edge from "act" back to "think" — that last edge is the loop.

Things to reuse, don't rewrite: your Doc07 model-calling function and your Doc07 tool functions — the whole point of this exercise is proving the graph gives the *same* behavior with the *same* tools, not building new tools.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Doc07's scratchpad (the growing message history) is exactly the kind of field that needs a reducer (`first_graph` 's Intermediate section) — every "think" and "act" step appends to it, and you don't want a later step's write to erase an earlier one.

The exact pieces:

- `class AgentState(TypedDict): task: str; scratchpad: Annotated[list[str], operator.add]; next_action: str` — `next_action` is what the conditional edge reads; it's a plain overwrite field, since only the *latest* decision matters.
- `def think(state: AgentState) -> dict:` — calls your Doc07 model function with `state["scratchpad"]`, decides "call a tool" or "finish," returns `{"scratchpad": [...new entry...], "next_action": "call_tool" or "finish"}`.
- `def act(state: AgentState) -> dict:` — runs whichever tool `think` picked (reusing your Doc07 tool functions), returns `{"scratchpad": [...the observation...]}`.
- `def route(state: AgentState) -> str:` return `state["next_action"]`
- `builder.add_conditional_edges("think", route, {"call_tool": "act", "finish": "finish_node"})`
- `builder.add_edge("act", "think")` — this is the loop: after acting, go back to thinking, exactly like the `while` loop's next iteration.
- A step limit: LangGraph graphs have a built-in `recursion_limit` (passed via `graph.invoke(..., config={"recursion_limit": N})`) that raises `GraphRecursionError` if exceeded — this is the graph's version of Doc07's manually-counted step limit, and you still need it here for the same reason you needed it there.

Think about what "same behavior" actually needs to mean here. It's not enough that the graph *runs* without crashing — the entire point of this exercise (per the README) is that it reaches the **same final answers** as your original loop, on the **same test prompts**. That's a real equivalence claim, and it needs a real check, not just eyeballing one output.

The real design question isn't just "does my graph work" — it's "how do I actually prove it behaves the same as the thing it's replacing?"

The extra piece that answers that:

- A small comparison harness: run the same list of test prompts through both your original Doc07 loop and your new graph, collect each one's final answer, and assert they match (or are equivalent in meaning, if the model's wording can legitimately vary). Log the step count from each too — a graph that reaches the right answer in *way* more steps than your original loop

might be hiding a routing bug (looping unnecessarily) even though the final answer happens to look right.

```
# agent_loop_to_graph_practice.py
test_prompts = [...] # the same prompts you used to validate the Doc07 loop

for prompt in test_prompts:
    loop_answer = run_original_loop(prompt)
    start = {"task": prompt, "scratchpad": [], "next_action": ""}
    graph_answer = graph.invoke(start)
    assert loop_answer == graph_answer["final_answer"], f"Mismatch: {prompt}"
```

Difference between Basic and Intermediate: Basic gets the graph itself built and running correctly. Intermediate also treats "gives the same answers as your original loop" as something to actually verify with a small automated check across your real test prompts, not just something to assume because the graph compiled and ran once without an error.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define state: task, scratchpad (accumulates), next_action, final_answer

make think: call the model with the scratchpad so far
    if the model wants a tool: record which one, set next_action = "call_tool"
    if the model is done: set next_action = "finish", save final_answer

make act: run whichever tool think picked, append the result to scratchpad

make finish_node: nothing more to do, just an end point

build the graph:
    start -> think
    from think, conditional:
        "call_tool" -> act
        "finish" -> finish_node
    act -> think    (this is the loop)
    finish_node -> end

run it with a real task, with a recursion limit set so it can't
run forever by mistake
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# agent_loop_to_graph_practice.py
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict):
    task: str
    scratchpad: Annotated[list[str], operator.add]
    next_action: str
    final_answer: str

def think(state):
    # reuse your Doc07 model-calling function here
```

```

decision = call_model_with_tools(state["task"], state["scratchpad"])
if decision["wants_tool"]:
    note = f"Thought: calling {decision['tool_name']}"
    return {"scratchpad": [note], "next_action": "call_tool"}
return {
    "scratchpad": ["Thought: done"],
    "next_action": "finish",
    "final_answer": decision["answer"],
}

def act(state):
    # reuse your Doc07 tool functions here
    result = run_chosen_tool(state)
    return {"scratchpad": [f"Observation: {result}"]}

def finish_node(state):
    return {}

```

Expected output if you run just this: nothing — add the conditional edge, the loop-back edge, `compile()`, and an `invoke(...)` call with a real task and a `recursion_limit` set.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```

from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

define AgentState with: task, scratchpad (accumulating),
    next_action, final_answer

define think(state) -> dict:
    call your Doc07 model function with state["task"] and state["scratchpad"]
    if it wants a tool: return scratchpad update + next_action="call_tool"
    else: return scratchpad update + next_action="finish" + final_answer

define act(state) -> dict:
    run the tool your Doc07 tool functions expose, using what think decided
    return the observation as a scratchpad update

define finish_node(state) -> dict:
    return {}

define route(state) -> str:
    return state["next_action"]

build:
    builder = StateGraph(AgentState)
    builder.add_node("think", think)
    builder.add_node("act", act)
    builder.add_node("finish_node", finish_node)
    builder.add_edge(START, "think")
    builder.add_conditional_edges(
        "think", route, {"call_tool": "act", "finish": "finish_node"},
    )
    builder.add_edge("act", "think")
    builder.add_edge("finish_node", END)

run:
    graph = builder.compile()
    starting_state = {
        "task": "What is 12 * 7, then search for that number?",
        "scratchpad": [], "next_action": "", "final_answer": "",
    }

```

```
result = graph.invoke(starting_state, config={"recursion_limit": 20})
print(result["final_answer"])
print(result["scratchpad"])
```

Trace one full cycle by hand: `think` decides a tool is needed, `act` runs it, control goes back to `think` — does it see the tool's result in `scratchpad` ? Then run it and compare against the [Solution](#).

Once that's working, prove it actually matches your original loop, not just that it runs:

```
reuse the exact same test prompts you used to validate your original Doc07 loop

for each test prompt:
    run it through your original loop -> loop_answer
    run it through the new graph -> graph_answer
    compare them -- do they match (or mean the same thing)?
    also compare step counts -- is the graph taking a similar number of steps?

report any mismatches clearly, naming which prompt failed and how
```

```
# agent_loop_to_graph_practice.py
test_prompts = [
    "What is 12 * 7?",
    "Search for the capital of France and tell me its population.",
    "What's 2 + 2?", # no tool actually needed
]

for prompt in test_prompts:
    loop_answer = run_original_loop(prompt) # your Doc07 loop, unchanged
    graph_result = graph.invoke(
        {
            "task": prompt,
            "scratchpad": [],
            "next_action": "",
            "final_answer": "",
        },
        config={"recursion_limit": 20},
    )
    graph_answer = graph_result["final_answer"]
    if loop_answer.strip() == graph_answer.strip():
        status = "MATCH"
    else:
        status = "MISMATCH"
    print(f"[{status}] {prompt!r}")
    if status == "MISMATCH":
```

```
print(f"  loop:  {loop_answer}")  
print(f"  graph: {graph_answer}")
```

Fill in `run_original_loop` (it's your actual Doc07 function — don't rewrite it) and run this against all your real test prompts before checking the [Solution](#).

Difference between Basic and Intermediate: Basic gets one graph built, wired, and running on one prompt. Intermediate runs a real side-by-side comparison against your original loop across every test prompt you already trust, which is the only way to actually back up the claim "this graph does the same thing my loop did" instead of just hoping it's true.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)